

Curso

Herramientas de Computación en la Nube

Clase 8
Febrero 14 de 2026

Git -Github

Taller práctico

*Este documento se fundamenta en los principios de arquitectura y sistemas de datos descritos en **Designing Data-Intensive Applications**, aplicados a un caso académico de analítica operativa. Consultar la bibliografía al final.*

Profesor
Alvaro Mauricio Montenegro Díaz
Universidad de la Sabana

Febrero 2026

Vamos a desarrollar un **Taller de Git y GitHub** pensado para:

- Científicos de datos
- Desarrolladores
- Arquitectos
- Personas que van a trabajar en plataformas reales

1. Historia y Filosofía

1.1. Antes de Git

En los 90 y 2000:

- CVS
- Subversion (SVN)

Modelo:

- Servidor central
- Sin repos locales completos
- Branching costoso
- Merging doloroso

Problema:

- No escalaban a proyectos grandes
- Poco tolerantes a colaboración masiva

1.2. Nace Git (2005)

Creador: Linus Torvalds

Motivación:

- Mantener el kernel de Linux
- Necesidad de sistema:
 - Distribuido
 - Rápido
 - Con branching barato

- Con integridad criptográfica

Idea revolucionaria

Cada desarrollador tiene el repositorio completo.

Git no guarda archivos.

Guarda snapshots del estado del proyecto.

En los dos siguientes secciones instalaremos Git en Mac y en Windows

2. Instalación de Git -MacacOS (Apple Silicon o Intel)

2.1. Verificar si ya está instalado

En la terminal:

```
git --version
```

Si responde algo como:

```
git version 2.43.0
```

Ya está instalado.

Si no, macOS puede mostrar:

The 'git' command requires the command line developer tools.

2.2. Opción A - Instalar vía Xcode Command Line Tools

Ejecuta en la terminal:

```
xcode-select --install
```

Se abrirá una ventana para instalar herramientas de desarrollo.

Luego verifica:

```
git --version
```

Esto es suficiente para la mayoría de usuarios.

2.3. Opción B (Profesional) — Instalar vía Homebrew

Si usas Homebrew (recomendado en entornos técnicos):

Primero verifica si tienes brew:

```
brew --version
```

Si no lo tienes, instálalo desde:

<https://brew.sh>

Luego:

```
brew install git
```

Verifica:

```
git --version
```

2.4. ¿Cuál usar?

Método	Cuándo usar
xcode-select	Curso estándar
brew	Entorno técnico profesional

Si vas a usar Python, Docker, Dask, etc., Homebrew es recomendable.

2.5. Configuración inicial obligatoria

Después de instalar Git:

```
git config --global user.name "Tu Nombre"
```

```
git config --global user.email "tu_email@dominio.com"
```

Verifica:

```
git config -list
```

o

```
git config -list --show-origin
```

para ver la configuración en internet

2.6. Confirmar archivo real

Puedes inspeccionar directamente el archivo:

```
cat ~/.gitconfig
```

Debes ver algo como

```
[user]
```

```
    name = Álvaro Mauricio Montenegro Díaz
```

```
    email = ammontenegrod@gmail.com
```

3. Instalación en Windows

3.1. Descargar Git

Ir a:

<https://git-scm.com/download/win>

Descargar instalador oficial.

3.2. Proceso de instalación (opciones recomendadas)

Durante la instalación selecciona estas opciones)

- ✓ Editor: VS Code (si está instalado) o Vim
- ✓ PATH: Git from the command line and also from 3rd-party software
- ✓ HTTPS backend: OpenSSL
- ✓ Line endings: Checkout Windows-style, commit Unix-style
- ✓ Terminal: Git Bash (recomendado)

Finalizar instalación.

3.3. Verificar instalación

Abrir Git Bash o PowerShell:

```
git --version
```

3.4. Configuración inicial obligatoria. Identidad global

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu_email@dominio.com"
```

Verificar:

```
git config --list
```

Debe mostrar algo como

```
user.name=Álvaro Mauricio Montenegro Díaz
user.email=ammontenegrod@gmail.com
...
```

4. Configuración recomendada adicional (macOS y Windows)

4.1. Cambiar rama principal por defecto

```
git config --global init.defaultBranch main
```

4.2. Activar colores en consola

```
git config --global color.ui auto
```

4.3. Ver configuración global

```
git config --global --list
```

posible salida

```
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
user.name=Alvaro Mauricio Montenegro Díaz
user.email=ammontenegrod@gmail.com
init.defaultbranch=main
color.ui=auto
```

4.4. Confirmar archivo real

```
cat ~/.gitconfig (MAC-Linux)
```

```
cat $HOME\.gitconfig      (Windows, PowerShell)
```

Deberias ver algo como

```
[filter "lfs"]
  process = git-lfs filter-process
  required = true
  clean = git-lfs clean -- %f
  smudge = git-lfs smudge -- %f
[user]
  name = Alvaro Mauricio Montenegro Díaz
  email = ammontenegrod@gmail.com
[init]
  defaultBranch = main
[color]
  ui = auto
```

4.5. El paso siguiente (muy importante) es configurar autenticación

En 2026 no se usa password plano.

Opciones:

- A) SSH (recomendado profesionalmente)
- B) Personal Access Token (PAT)

5. OPCIÓN A – SSH (RECOMENDADA PROFESIONALMENTE)

5.1. MAC – Configurar SSH

5.1.1 Verificar si ya tienes clave

```
ls -al ~/.ssh
```

Si ves:

```
id_ed25519
id_ed25519.pub
```

Ya tienes una.

5.1.2 Crear nueva clave (si no existe)

```
ssh-keygen -t ed25519 -C "tu\_email@dominio.com"
```

Presiona Enter para aceptar ubicación por defecto:

Puedes dejar passphrase vacía o agregar una.

5.1.3 Iniciar agente SSH

```
eval "$(ssh-agent -s)"
```

Agregar clave

```
ssh-add ~/.ssh/id_ed25519
```

5.1.4 Copiar clave pública

```
cat ~/.ssh/id_ed25519.pub
```

Copia todo el texto (empieza por ssh-ed25519).

5.1.5 Agregar a GitHub

Ve a tu cuenta de Github y sigue esta ruta allí

[GitHub](#) → [Settings](#) → [SSH and GPG keys](#) → [New SSH Key](#)

Pega la clave

5.1.6 Probar conexión

```
ssh -T git@github.com
```

Debe decir

```
Hi usuario! You've successfully authenticated.
```

5.1.7 Cambiar remoto a SSH

Para hacer esto debes crear un directorio que será luego un repositorio. En el ejemplo de clase será: [pqr-hybrid-lakehouse](#)

Crea el directorio y pásate a el.

```
mkdir pqr-hybrid-lakehouse
```

```
cd pqr-hybrid-lakehouse
```

convierte el directorio en un repositorio de Git:

```
git .init
```

esto crea una archivo .git.

Para visualizarlo escribe

```
ls -la
```

Debes ver

```
.git
```

Estamos listos para cambiar a remoto. Ahora puedes hacer

```
git remote set-url origin git@github.com:usuario/repo.git
```

6. Flujo correcto típico para crear un proyecto nuevo

```
mkdir pqr-hybrid-lakehouse
```

```
cd pqr-hybrid-lakehouse
```

```
git init
```

```
echo "# PQR" > README.md
```

```
git add README.md
```

```
git commit -m "feat: initial commit"
```

```
git remote add origin git@github.com:usuario/pqr-hybrid-lakehouse.git
```

```
git push -u origin main
```

7. Si quieres que el repo esté en la organización

Debes:

Crear el repositorio en GitHub dentro de la organización.

En GitHub:

- Ve a la organización, digamos **AprendizajeProfundo**
- Click en “New repository”
- Nombre: pqr-hybrid-lakehouse
- No agregar README (porque ya lo tienes local)
- Crear

Reemplaza por el nombre exacto del repo que creaste en la org:

```
echo "# pqr-hybrid-lakehouse" >> README.md
git init
git add README.md
git commit -m "primer commit"
git branch -M main
git remote set-url origin git@github.com:AprendizajeProfundo/pqr-hybrid-lakehouse.git
```

Verifica

```
git remote -v
```

debes reponder

```
origin git@github.com:AprendizajeProfundo/pqr-hybrid-lakehouse.git (fetch)
origin git@github.com:AprendizajeProfundo/pqr-hybrid-lakehouse.git (push)
```

Asegúrate de que tu rama local se llama main

verifica

```
git branch
```

Si ves * master, renómbrala:

```
git branch -M main
```

Ahora sí:

```
git push -u origin main
```

8. Problemas comunes y soluciones

8.1. “git not recognized”

En Windows:

- Reiniciar terminal
- Verificar que Git esté en PATH

8.2. Permisos SSH incorrectos (Mac)

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/id_ed25519
```

9. Creación de un archivo .gitignore

Debes crear un archivo .gitignore, como práctica profesional seria.

Especialmente porque:

- Estás en Mac o en Windows y hay archivos del S.O. que no deben ir al repositorio de Github, por ejemplo *.DS_Store* de MAC o *Desktop.ini* de Windows.
- Usarás Python
- Docker
- Posiblemente Jupyter
- Archivos temporales
- Archivos de sistema
- Datos lakehouse

9.1. 🧠 ¿Qué es .gitignore?

Es un archivo que le dice a Git:

“Estos archivos nunca deben versionarse.”

Git ignora esos patrones automáticamente.

9.2. Para tu proyecto PQR

Debemos cubrir:

- macOS
- Windows
- Python
- Docker
- IDEs
- Data engineering
- Lakehouse
- Logs

- Entornos virtuales

9.3. .gitignore profesional para tu proyecto

Crea archivo:

```
touch .gitignore
```

Contenido recomendado:

```
#####  
# OS  
#####  
.DS_Store  
Thumbs.db  
  
#####  
# Python  
#####  
__pycache__/  
*.pyc  
*.pyo  
*.pyd  
*.log  
*.sqlite3  
  
#####  
# Virtual environments  
#####  
venv/  
.env/  
.envrc  
*.env  
  
#####  
# Jupyter  
#####
```

.ipynb_checkpoints/

#####

Data / Lakehouse

#####

data/

lakehouse/

raw/

bronze/

silver/

gold/

*.parquet

*.jsonl

#####

Docker

#####

*.pid

*.sock

#####

Prefect

#####

.prefect/

.prefectignore

#####

Logs

#####

logs/

*.log

#####

IDE

```
#####

.vscode/
.idea/

#####

# System temp
#####

tmp/
temp/
*.swp

#####

# Secrets
#####

.env.local
.env.production
*.key
*.pem
```

9.4. Explicación clave para tu arquitectura

Nunca versionar:

- Parquet
- JSONL raw
- Datasets
- Logs
- Claves privadas
- Archivos generados por pipelines

Eso debe vivir en:

- RustFS
- S3

- Storage externo

9.5. Si ya agregaste .DS_Store

Si Git ya lo está rastreando:

```
git rm --cached .DS_Store
```

Luego:

```
git commit -m "chore: remove system files"
```

```
git push
```

9.6. Verificar qué archivos están siendo ignorados

```
git status --ignored
```

9.7. Mejora profesional adicional con .gitattributes

Crear un archivo .gitattributes para normalizar line endings:

```
touch .gitattributes
```

Contenido:

```
* text=auto
*.py text eol=lf
*.sh text eol=lf
*.yaml text eol=lf
*.yml text eol=lf
*.toml text eol=lf
*.json text eol=lf
*.md text eol=lf
```

* *text=auto* evita problemas Windows/Mac.

Las siguientes líneas le están diciendo a Git:

“Estos archivos son texto y deben guardarse en el repositorio con final de línea LF (Unix).”

Problema clásico Windows vs Unix: Windows usa:

CRLF → \r\n

Linux – MacOS usan

LF → \n

Si no controlas esto:

- Git mostrará cambios falsos
- Docker puede fallar
- Scripts .sh no ejecutan

- CI/CD rompe
- Dask workers pueden fallar en Linux
- Prefect en contenedor puede fallar

10. Arquitectura limpia implica:

Repositorio → código + contratos + documentación

Datos → RustFS

Infraestructura → Docker

Credenciales → variables de entorno

11. Recomendación para tu curso

Haz que los estudiantes:

1. Creen `.gitignore`
2. Hagan commit limpio
3. Comprendan por qué no versionar datos

12. Flujo estándar para actualizar repo

12.1. Ver estado actual

Siempre comienza así:

```
git status
```

Te muestra:

- Archivos modificados
- Archivos nuevos
- Archivos eliminados
- Rama actual

12.2. Agregar cambios al staging

Opcion simple (todo)

```
git add .
```

Esto agrega:

- Nuevos archivos
- Modificados
- Eliminados

Solo funciona correctamente si no estás fuera del root del repo.

Opción profesional (recomendado)

Ver qué vas a agregar:

```
git status
```

Agregar selectivamente

```
git add archivo1.md
```

```
git add carpeta/
```

12.3. Confirmar Lo que vas a hacer commit

```
git status
```

verás

```
Changes to be committed:
```

12.4. Crear commit

```
git commit -m "feat: actualiza arquitectura y data-contracts"
```

Tipo	Uso
feat	Nueva funcionalidad
fix	Corrección
docs	Documentación
refactor	Cambio estructural
chore	Tareas técnicas
test	Pruebas

Subir a Github

```
git push
```

Si es al priemra vez en esa rama

```
git push -u origin main
```

12.5. Flujo completo resumido

```
git status
```

```
git add .
```

```
git commit -m "prefijo: mensaje claro"
```

```
git push
```

12.6. Ver historial

```
git log --oneline --graph --decorate
```

13. ¿Qué es Git LFS?

Git **no fue diseñado para archivos grandes** (binarios pesados):

- PDFs grandes
- Imágenes de alta resolución
- Modelos ML
- Datasets
- Archivos .zip

Problema:

Git guarda snapshots completos.

Un archivo grande repetido en muchos commits hace que el repo crezca exponencialmente.

Git LFS resuelve eso.

13.1. ¿Qué hace Git LFS?

En vez de guardar el archivo real en Git:

- Guarda un **puntero pequeño**
- El archivo real se guarda en almacenamiento separado
- GitHub tiene soporte nativo para LFS

13.2. Ahora expliquemos la configuración lfs, si ya se tiene

Si *.gitconfig* muestra:

```
[filter "lfs"]  
    process = git-lfs filter-process  
    required = true  
    clean = git-lfs clean -- %f  
    smudge = git-lfs smudge -- %f
```

Esto define cómo Git interactúa con archivos LFS.

13.3. Explicación línea por línea

13.3.1 **process = git-lfs filter-process**

Este es el mecanismo moderno.

Git ejecuta un proceso continuo (git-lfs filter-process) que:

- Intercepta archivos marcados como LFS
- Reemplaza contenido por punteros
- Descarga el archivo real cuando haces checkout

Es más eficiente que el método antiguo.

13.3.2 **required = true**

Significa:

Si el filtro LFS falla, el commit falla.

Evita corrupción de repositorio.

Es buena práctica.

13.3.3 **clean** = **git-lfs clean** – %f

Este comando se ejecuta cuando haces:

```
git add archivo_grande.bin
```

Lo que hace:

- Toma el archivo real
- Lo sube a almacenamiento LFS
- Reemplaza el contenido por un puntero pequeño

Ejemplo del puntero:

```
version https://git-lfs.github.com/spec/v1
oid sha256:abc123...
size 104857600
```

Ese texto es lo que Git realmente guarda.

13.3.4 **smudge** = **git-lfs smudge** – %f

Se ejecuta cuando haces:

```
git checkout
```

Hace lo contrario:

- Lee el puntero
- Descarga el archivo real desde LFS
- Lo coloca en tu carpeta de trabajo

13.3.5 **Resumen del flujo**

Acción	Qué pasa
git add	clean reemplaza archivo por puntero
git push	Archivo real va a almacenamiento LFS
git clone	Descarga punteros
git checkout	smudge descarga archivo real

13.4. ¿Por qué ya lo tienes configurado, si te salió el mensaje?

Porque:

- Instalaste Git LFS en algún momento
- O Homebrew lo activó
- O algún repo inicializó LFS

Puedes verificar:

```
git lfs version
```

13.5. ¿Es bueno tenerlo?

Sí, pero con criterio.

Para tu proyecto PQR:

No debes usar LFS para:

- Parquet generados
- Datasets
- Archivos de salida

Esos NO deben versionarse.

LFS es útil para:

- Modelos ML grandes
- Recursos estáticos necesarios

13.6. Riesgo común

Si subes datasets a Git LFS:

- GitHub tiene límites de almacenamiento
- Puede volverse costoso
- No es práctica correcta en data engineering

Para datos:

S3 (RustFS)

No Git

13.7. Si quieres saber qué archivos usan LFS

En un repo:

```
git lfs track
```

Para ver archivos ya en LFS:

```
git lfs ls-files
```

13.8. Recomendación

Para el proyecto:

- ✓ Mantener LFS instalado
- ✓ No usarlo para datos lakehouse
- ✓ Documentar en README que datasets no van en Git

14. Problemas comunes y soluciones

14.1. “git not recognized”

En Windows:

- Reiniciar terminal
- Verificar que Git esté en PATH

14.2. Permisos SSH incorrectos (Mac)

```
chmod 700 ~/.ssh
```

```
chmod 600 ~/.ssh/id_ed25519
```

15. Resultado esperado

Después de esta guía, el estudiante debe poder:

- Ejecutar `git --version`
- Hacer `git init`
- Hacer commits locales
- Configurar identidad
- Prepararse para GitHub

16. Anexo 1: Conceptos Fundamentales

16.1. Repositorio

Base de datos local que contiene:

- Commits
- Árboles
- Objetos
- Referencias

Puede ser:

- Local
- Remoto
- Ambos

16.2. Commit

Un commit es:

- Snapshot del proyecto
- Con autor
- Timestamp
- Mensaje
- Hash SHA-1/SHA-256

16.3. Ejemplo:

a3f8b2c... (hash)

Es inmutable.

16.4. Branch

Un puntero móvil a una secuencia de commits.

- main
- dev
- feature/dask-pipeline

Branching es barato.

16.5. Remote

Referencia a otro repositorio.

Generalmente:

- origin

16.6. Clone

`git clone https://github.com/user/repo.git`

Clona TODO:

- Historia completa
- Todas las ramas

- Todos los objetos

No es copiar archivos.

Es copiar la base de datos Git.

16.7. Fork (GitHub, no Git)

Fork es concepto de GitHub:

- Copia el repo a tu cuenta
- Mantiene relación con el original
- Permite Pull Requests

Se usa cuando:

- No tienes permisos directos
- Proyecto open-source

17. Anexo 2: Flujo Profesional Local

17.1. Crear proyecto nuevo

Crea un directorio para el nuevo proyecto

```
mkdir pqr-hybrid-lakehouse  
cd pqr-hybrid-lakehouse  
git init
```

Crear archivo:

```
echo "# PQR Hybrid Lakehouse" > README.md  
git add README.md  
git commit -m "feat: initial commit"
```

17.2. Conectar a GitHub

Crear repo vacío en GitHub.

Luego:

```
git remote add origin https://github.com/usuario/pqr-hybrid-lakehouse.git
git branch -M main
git push -u origin main
```

17.3. Ciclo diario profesional

```
git status
git add .
git commit -m "feat: add bronze schema"
git push
```

18. Anexo 3: Estrategia de Ramas Profesional

18.1. Modelo recomendado:

```
main      -> estable
dev       -> integración
feature/* -> trabajo específico
```

18.2. Ejemplo:

```
git checkout -b feature/dask-bronze
```

Trabajas.

```
git add .
git commit -m "feat: implement bronze pipeline"
git push origin feature/dask-bronze
```

Luego Pull Request.

19. Anexo 4: Credenciales y Seguridad

19.1. No usar usuario/contraseña

GitHub ya no acepta password directo.

19.2. Opción A — Personal Access Token (PAT)

1. GitHub → Settings → Developer Settings → Tokens
2. Generar token
3. Usarlo como password

19.3. Opción B — SSH (Profesional)

Generar clave:

```
ssh-keygen -t ed25519 -C "tu_email@dominio.com"
```

Agregar al agente:

```
eval "$(ssh-agent -s)"
```

```
ssh-add ~/.ssh/id_ed25519
```

Copiar clave pública:

```
cat ~/.ssh/id_ed25519.pub
```

Pegar en GitHub → SSH Keys.

Luego usar:

```
git@github.com:usuario/repo.git
```

Más seguro y profesional.

20. Anexo 5: Qué es Local y qué es Nube

Acción	Local GitHub	
Crear commit	✓	✗
Crear branch	✓	✗
Hacer push	✗	✓
Pull Request	✗	✓
Issues	✗	✓
Releases	✗	✓

Git = sistema

GitHub = plataforma colaborativa

21. Anexo 6: Pull Request Profesional

Un PR debe tener:

- Descripción clara
- Qué cambia
- Por qué
- Cómo probarlo
- Checklist

No es “ya subí cosas”.

21.1. Anexo 7: — Releases y Versionado

Versionado semántico:

v0.1.0 -> baseline

v0.2.0 -> Dask ready

v1.0.0 -> MVP estable

Crear tag:

```
git tag -a v0.1.0 -m "Baseline architecture"
```

```
git push origin v0.1.0
```

22. Anexo 8 – Fork Workflow (Open Source)

1. Fork en GitHub

2. Clonar tu fork

3. Agregar upstream:

```
git remote add upstream https://github.com/original/repo.git
```

4. Mantener sincronizado:

```
git fetch upstream
```

```
git merge upstream/main
```

23. Anexo 9: Proyectos, Issues y Boards

En GitHub:

- Issues = tareas
- Projects = tablero Kanban

- Milestones = hitos
- Labels = categorización

Arquitectura profesional no vive solo en código.

24. Anexo 10: Buenas Prácticas

- ✓ Commits pequeños y claros
- ✓ Mensajes semánticos
- ✓ No subir datos pesados
- ✓ No subir .env
- ✓ No subir credenciales
- ✓ Usar .gitignore serio
- ✓ Tags para versiones

25. Anexo 11: Manejo de tags

Tarea de curso